Document number: P0818R0
Date: 2017-10-16
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2014
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "tentatively ready" Issues for the November, 2017 (Albuquerque) meeting

Section references in this document reflect the section numbering of document WG21 N4659.

2305. Explicit instantiation of constexpr or inline variable template

Section: 17.7.2 [temp.explicit] **Status:** tentatively ready **Submitter:** John Spicer **Date:** 2016-07-14

According to 17.7.2 [temp.explicit] paragraph 1,

> An explicit instantiation of a function template or member function of a class template shall not use the `inline` or `constexpr` specifiers.

Should this apply to explicit specializations of variable templates as well?

(See also issues 1704 and 1728).

Proposed resolution (August, 2017):

Change 17.7.2 [temp.explicit] paragraph 1 as follows:

> A class, function, variable, or member template specialization can be explicitly instantiated from its template. A member function, member class or static data member of a class template can be explicitly instantiated from the member definition associated with its class template. An explicit instantiation of a function template ~~or~~**,** member function of a class template**, or variable template** shall not use the `inline` or `constexpr` specifiers.

2307. Unclear definition of “equivalent to a nontype template parameter”

Section: 17.6.2.1 [temp.dep.type] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-07-21

The description of whether a template argument is equivalent to a template parameter in 17.6.2.1 [temp.dep.type] paragraph 3 is unclear as it applies to non-type template parameters:

> A template argument that is equivalent to a template parameter (i.e., has the same constant value or the same type as the template parameter) can be used in place of that template parameter in a reference to the current instantiation. In the case of a non-type template argument, the argument must have been given the value of the template parameter and not an expression in which the template parameter appears as a subexpression.

For example:

```
template<int N> struct A {
  typedef int T[N];
  static const int AlsoN = N;
  A<AlsoN>::T s; // #0, clearly supposed to be OK
  static const char K = N;
  A<K>::T t;     // #1, OK?
  static const long L = N;
  A<L>::T u;     // #2, OK?
  A<(N)>::T v;   // #3, OK?
  static const int M = (N);
  A<M>::T w;     // #4, OK?
};
```

#1 narrows the template argument. This obviously should not be the injected-class-name, because `A<257>::T` may well be `int[1]` not `int[257]` . However, the wording above seems to treat it as the injected-class-name.

#2 is questionable: there is potentially a narrowing conversion here, but it doesn't actually narrow any values for the original template parameter.

#3 is hard to decipher. On the one hand, this is an expression involving the template parameter. On the other hand, a parenthesized expression is specified as being equivalent to its contained expression.

#4 should presumably go the same way that #3 does.

Proposed resolution (August, 2017):

Change 17.6.2.1 [temp.dep.type] paragraph 3 as follows:

> A template argument that is equivalent to a template parameter ~~(i.e., has the same constant value or the same type as the template parameter)~~ can be used in place of that template parameter in a reference to the current instantiation. **For a template *type-parameter*, a template argument is equivalent to a template parameter if it denotes the same type. For a non-type template parameter, a template argument is equivalent to a template parameter if it is an *identifier* that names a variable that is equivalent to the template parameter. A variable is equivalent to a template parameter if**
>
> - **it has the same type as the template parameter (ignoring cv-qualification) and**
> - **its initializer consists of a single *identifier* that names the template parameter or, recursively, such a variable.**
>
> **[*Note:* Using a parenthesized variable name breaks the equivalence. —*end note*]** ~~In the case of a non-type template argument, the argument must have been given the value of the template parameter and not an expression in which the template parameter appears as a subexpression.~~ [*Example:*
>
> ```
> template <class T> class A {
>   A* p1;                  // A is the current instantiation
>   A<T>* p2;               // A<T> is the current instantiation
>   A<T*> p3;               // A<T*> is not the current instantiation
> ```

```
  ::A<T>* p4;               // ::A<T> is the current instantiation
  class B {
    B* p1;                  // B is the current instantiation
    A<T>::B* p2;            // A<T>::B is the current instantiation
    typename A<T*>::B* p3; // A<T*>::B is not the current instantiation
  };
};

template <class T> class A<T*> {
  A<T*>* p1;                // A<T*> is the current instantiation
  A<T>* p2;                 // A<T> is not the current instantiation
};

template <class T1, class T2, int I> struct B {
  B<T1, T2, I>* b1;         // refers to the current instantiation
  B<T2, T1, I>* b2;         // not the current instantiation
  typedef T1 my_T1;
  static const int my_I = I;
  static const int my_I2 = I+0;
  static const int my_I3 = my_I;
  static const long my_I4 = I;
  static const int my_I5 = (I);
  B<my_T1, T2, my_I>* b3;   // refers to the current instantiation
  B<my_T1, T2, my_I2>* b4;  // not the current instantiation
  B<my_T1, T2, my_I3>* b5;  // refers to the current instantiation
  B<my_T1, T2, my_I4>* b6;  // not the current instantiation
  B<my_T1, T2, my_I5>* b7;  // not the current instantiation
};
```

—*end example*]

2313. Redeclaration of structured binding reference variables

Section: 11.5 [dcl.struct.bind] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-08-12

According to the current rules for structured binding declarations, the user-defined case declares the bindings as variables of reference type. This presumably makes an example like the following valid:

```
auto [a] = std::tuple<int>(0);
extern int &&a; // ok, redeclaration, could even be in a different TU
```

This seems unreasonable, especially in light of the fact that it only works for the user-defined case and not the built-in case (where the bindings are not modeled as references).

Proposed resolution (August, 2017):

Change 11.5 [dcl.struct.bind] paragraph 3 as follows:

> Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. The *unqualified-id* `get` is looked up in the scope of `E` by class member access lookup (6.4.5 [basic.lookup.classref]), and if that finds at least one declaration, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)` where `get` is looked up in the associated namespaces (6.4.2 [basic.lookup.argdep]). In either

case, `get<i>` is interpreted as a *template-id*. [*Note:* Ordinary unqualified lookup (6.4.1 [basic.lookup.unqual]) is not performed. —*end note*] In either case, `e` is an lvalue if the type of the entity `e` is an lvalue reference and an xvalue otherwise. Given the type T_i designated by `std::tuple_element<i, E>::type`, ~~each v_i is a variable~~ **variables are introduced with unique names r_i** of type "reference to T_i" initialized with the initializer **(11.6.3 [dcl.init.ref])**, where the reference is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise**. Each v_i is the name of an lvalue of type T_i that refers to the object bound to r_i**; the referenced type is T_i.

2315. Whst is the "corresponding special member" of a variant member?

Section: 15.8 [class.copy] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-08-15

According to 15.8.1 [class.copy.ctor] bullet 10.1,

> A defaulted copy/move constructor for a class `X` is defined as deleted (11.4.3 [dcl.fct.def.delete]) if `X` has:
>
> - a variant member with a non-trivial corresponding constructor and `X` is a union-like class,

However, it is not clear from this specification how to handle an example like:

```
struct A {
  A();
  A(const A&);
};
union B {
  A a;
};
```

since there is no corresponding special member in `A`.

Proposed resolution (August, 2017):

Change 15.8.1 [class.copy.ctor] paragraph 10 as follows:

> An implicitly-declared copy/move constructor is an inline public member of its class. A defaulted copy/move constructor for a class `X` is defined as deleted (11.4.3 [dcl.fct.def.delete]) if `X` has:
>
> - ~~a variant member with a non-trivial corresponding constructor and `X` is a union-like class,~~
> - a potentially constructed subobject type `M` (or array thereof) that cannot be copied/moved because overload resolution (16.3 [over.match]), as applied to find `M`'s corresponding constructor, results in an ambiguity or a function that is deleted or inaccessible from the defaulted constructor,
> - **a variant member whose corresponding constructor as selected by overload resolution is non-trivial,**
> - any potentially constructed subobject of a type with a destructor that is deleted or inaccessible from the defaulted constructor, or,

- for the copy constructor, a non-static data member of rvalue reference type.

2338. Undefined behavior converting to short enums with fixed underlying types

Section: 8.2.9 [expr.static.cast] **Status:** tentatively ready **Submitter:** Thomas Köppe **Date:** 2017-03-24

The specifications of `std::byte` (21.2.5 [support.types.byteops]) and `bitmask` (20.4.2.1.4 [bitmask.types]) have revealed a problem with the integral conversion rules, according to which both those specifications have, in the general case. undefined behavior. The problem is that a conversion to an enumeration type has undefined behavior unless the value to be converted is in the range of the enumeration.

For enumerations with an unsigned fixed underlying type, this requirement is overly restrictive, since converting a large value to an unsigned integer type is well-defined.

Proposed resolution (August, 2017):

Change 8.2.9 [expr.static.cast] paragraph 10 as follows:

> A value of integral or enumeration type can be explicitly converted to a complete enumeration type. ~~The~~ **If the enumeration type has a fixed underlying type, the value is first converted to that type by integral conversion, if necessary, and then to the enumeration type. If the enumeration type does not have a fixed underlying type, the** value is unchanged if the original value is within the range of the enumeration values (10.2 [dcl.enum]). ~~Otherwise~~**, and otherwise**, the behavior is undefined.